MagicBox Playbook Reference

Reference for every Ansible recipe in this repository: purpose, variables, commands, tags, prerequisites, verification and operational risks. It matches the code at the time of the build; `tools/check_docs.py` fails if a role, variable or tag is missing from this document.

Pinned toolchain: ansible-core 2.21.5 (Python 3.14 `.venv` managed by uv), community.general 13.5.0, community.docker 5.4.0, community.crypto 3.5.1.

1. Overview

Item	Value
Playbooks	playbooks/site.yml (make deploy: host, Portainer, every stack, apps), playbooks/status.yml (read-only status), playbooks/update.yml (OS packages)
Inventory	inventory/hosts.yml (git-ignored; template inventory/hosts.example.yml)
Group	magicbox
Project policy	inventory/group_vars/magicbox/main.yml
Secrets	inventory/group_vars/magicbox/vault.yml (ansible-vault, git-ignored)
Apps registry	inventory/group_vars/magicbox/apps.yml (magicbox_apps, git-ignored; template inventory/apps.example.yml)
Collections	requirements.yml, installed to .ansible/collections
Result	Portainer CE at https://magicbox.local:9443/ with environment homelab; every stack; gateway at https://magicbox.local/; registry at magicbox.local:5000; LAN only

`playbooks/site.yml` is the whole installation, run by `make deploy` and idempotent (ADR-0021). Its plays, in order:

1. *Configure MagicBox* (`become: true`): roles `preflight`, `mdns`, `firewall`, `docker`, `portainer`, `portainer_init`.
2. *Verify Portainer from the Ansible controller*: queries Portainer over the LAN.
3. *Deploy the stacks through Portainer*: role `stacks` with every stack in `stacks_enabled` (default stacks, RustFS, registry, Jenkins, Kong with the app routes).
4. *Register the apps in Jenkins*: `tasks/apps.yml`, i.e. `tasks/apps_jenkins.yml` (when `jenkins` is enabled) and the report `tasks/apps_report.yml`.

`playbooks/update.yml` has one play, *Update MagicBox packages*, that runs the `os_update` role. It needs the vault password only for `vault_become_password`.

2. Commands

Goal	Command
Create the environment	make setup
All local checks	make check
Container converge + idempotency test	make test-container (host), ARGS="--stacks all" (everything)
First deployment	make deploy EXTRA="-e magicbox_confirm_destructive=true"
Routine deployment (also after a new app, vault change or certificate renewal)	make deploy
Only one area	make deploy EXTRA="--tags docker"
Only the stacks and apps	make deploy EXTRA="--tags stacks,apps"
Selected stacks only	uv run ansible-playbook playbooks/site.yml --ask-vault-pass --tags stacks -e '{"stacks_enabled": ["postgresql"]}'

Goal	Command
Dry run with diff	uv run ansible-playbook playbooks/site.yml --ask-vault-pass --check --diff
Verification only (host, Portainer, stacks)	make deploy EXTRA="--tags verify"
Stack status (read-only)	make status
Edit the vault	make vault-edit
OS package update	make update
OS update + reboot if required	make update EXTRA="-e os_update_reboot=true"
Idempotency on MagicBox	uv run tools/idempotency_check.py playbooks/site.yml --vault-password-file <file>
List tasks / tags	uv run ansible-playbook playbooks/site.yml --list-tasks / --list-tags

`make deploy`, `verify`, `status` and `update` ask for the vault password; the sudo password is `vault_become_password` (`ansible_become_password` in `inventory/hosts.yml`). `BECOME=-K` prompts for it instead.

A `--check` run cannot complete the `docker` and `portainer` roles on a fresh host because the packages they verify are not installed yet.

3. Tags

Tag	Selects
preflight	The preflight role
always	Also on preflight: it runs with any --tags selection, so checks and facts are always present
mdns	The mdns role
firewall	The firewall role
docker	The docker role
portainer	The portainer and portainer_init roles, including TLS tasks
portainer-init	Only the portainer_init role (site environment)
stacks	The stacks role (every stack in stacks_enabled, Kong app routes included)
apps	The apps play: apps.json, Jenkins app jobs, report (needs the jenkins and kong stacks deployed)
status	The read-only status report (in playbooks/status.yml)
update	The os_update role (in playbooks/update.yml)
tls	Only TLS provisioning inside portainer (certificate renewal, CA fetch)
verify	Verification tasks in all roles plus the controller play, stack checks included
controller-verify	Only the controller play; skipped by the container test

`--tags verify` runs `preflight` (tagged `always`) and every `verify` task. It changes nothing on a converged host.

4. Shared variables

Defined in `inventory/hosts.yml` (environment facts) or `inventory/group_vars/magicbox/main.yml` (project policy).

Variable	Default	Description
magicbox_lan_interface	required	LAN interface on MagicBox (ip -br link). Avahi publishes only here.
magicbox_lan_cidr	required	Private IPv4 LAN subnet allowed to reach SSH, Portainer and mDNS.

Variable	Default	Description
ansible_ssh_extra_args	"-4"	SSH over IPv4: ufw allows only the IPv4 magicbox_lan_cidr; an IPv6 link-local connection is refused by preflight.
ansible_become_password	{{ vault_become_password }}	sudo password of ansible_user, set in inventory/hosts.yml; remove with passwordless sudo.
vault_become_password	required (vault)	sudo password; anyone with the vault password can become root on MagicBox.
ansible_become_exe	sudo	Ubuntu 26.04: sudo.ws (classic sudo; sudo-rs prompts time out, ADR-0002).
magicbox_hostname	magicbox	Hostname and mDNS name (<name>.local); single DNS label.
magicbox_expected_architecture	x86_64	Asserted CPU architecture.
magicbox_supported_os	Ubuntu 24.04/26.04, Debian 12/13	Asserted OS/version map (ADR-0002).
magicbox_confirm_destructive	false	Allows destructive steps for one run (ADR-0008). Pass with -e.
magicbox_site_name	homelab	Name of the Portainer environment (site) for the local Docker engine (ADR-0010).
magicbox_portainer_api_url	https://magicbox.local:9443	Portainer API used by portainer_init and stacks; keep in sync with portainer_https_port.
magicbox_portainer_ca_cert	/opt/magicbox/portainer/certs/ca.crt	CA used to validate the API; keep in sync with portainer_cert_dir.
portainer_admin_password	{{ vault_portainer_admin_password }}	Bootstrap admin password, from the vault.
vault_portainer_admin_password	required (vault)	At least 12 characters; in vault.yml only.
vault_postgresql_password, vault_rabbitmq_password, vault_grafana_admin_password	required for the matching stack (vault)	At least 12 characters each.
vault_openwebui_admin_password, vault_openwebui_secret_key	required for the ollama stack (vault)	At least 12 characters each; Open WebUI admin password (first start only) and session signing key.
vault_jenkins_admin_password, vault_jenkins_github_token, vault_jenkins_portainer_token	Jenkins stack (vault)	Admin password (12+ characters) required; tokens optional.
magicbox_apps	[] (inventory/group_vars/magicbox/apps.yml)	Apps (ADR-0016, ADR-0017): name (Jenkins job, Portainer stack, container, URL path /<name>), port (host port on 127.0.0.1, 18000-18999), repo (https://github.com/<owner>/<repository>). Maintained by tools/new_app.py.
vault_rustfs_access_key, vault_rustfs_secret_key	required for the RustFS stack (vault)	At least 12 characters each; RustFS root credentials (console login, S3 keys), applied at every start.

Variable	Default	Description
vault_kong_ollama_api_key	required for the Kong stack (vault)	At least 12 characters; API key for https://magicbox.local/ollama/ (header or query parameter apikey).
vault_dkron_password	required for the Kong stack (vault)	At least 12 characters; login admin of https://magicbox.local/dkron/ (Kong basic-auth).
vault_registry_password	required for the registry stack (vault)	At least 12 characters; registry login of stacks_registry_user and Jenkins credential registry.

5. Role `preflight`

Purpose. Fail fast on unsupported or misconfigured hosts and prepare the baseline: OS/architecture/inventory/vault assertions, LAN CIDR validation, detection of where the SSH controller connects from, hostname, base packages.

Prerequisites. `magicbox.local` resolves from the controller (Avahi installed during the manual steps); SSH key login as a sudo-capable user; Python 3 on the host (present on Ubuntu/Debian server installs).

Variable	Default	Description
preflight_base_packages	ca-certificates, curl, gnupg, iproute2, python3-cryptography, python3-debian, python3-requests	Packages needed by later roles and modules.
preflight_manage_hostname	true	Set the static hostname and 127.0.1.1 entry. false only in the test container.

Facts set. `preflight_controller_location`: `inside`, `outside` or `unknown` (no `SSH_CLIENT`, e.g. the container test), consumed by `firewall`.

Verification. All assertions run on every play (tag `always`).

Risks. Changing `magicbox_hostname` renames the host and the mDNS name; certificates must match (`portainer_tls_dns_names` follows automatically, the existing CA's name constraints do not: rotate the CA, installation guide 8.6).

6. Role `mdns`

Purpose. Publish `magicbox.local` on the LAN with Avahi (ADR-0003). MagicBox has no fixed IP: Avahi re-announces the name whenever DHCP changes the address. Avahi is first installed by hand (installation guide 1.5) so Ansible can connect by name; this role then owns its configuration. Installs `mdns_packages`, disables systemd-resolved's mDNS/LLMNR responders when systemd-resolved is installed, templates `/etc/avahi/avahi-daemon.conf` (host name, `allow-interfaces`, no AAAA records unless `mdns_use_ipv6`, no HINFO/workstation publishing, no reflector, no wide-area), and enables `avahi-daemon`.

Variable	Default	Description
mdns_use_ipv6	**false**	**Publish AAAA records (use-ipv6, publish-aaaa-on-ipv4). Off because the firewall and Kong accept only the IPv4 LAN; a link-local AAAA makes browsers fail with "address unreachable".**
mdns_packages	**avahi-daemon, avahi-utils, libnss-mdns**	**Responder, CLI tools, NSS module for local resolution.**

Handlers. *Restart systemd-resolved*, *Restart avahi-daemon*.

Verification. `avahi-resolve-host-name -4 magicbox.local` must return an address (retries 5 x 2 s);

with `mdns_use_ipv6: false`, `-6` must return none.

Risks. Avahi renames the host to `magicbox-2.local` if another device already claims the name; TLS then fails the name check. Restricting to `magicbox_lan_interface` means the name is invisible on other interfaces.

7. Role `firewall`

Purpose. Enable ufw with default deny inbound and allow only SSH, `firewall_lan_tcp_ports` and `firewall_lan_udp_ports` from `magicbox_lan_cidr` (ADR-0004). Rules persist across reboots.

Variable	Default	Description
firewall_ssh_port	22	SSH port allowed from the LAN.
firewall_lan_tcp_ports	[9443, 80, 443]	TCP ports allowed from the LAN (Portainer; Kong API gateway, host networking).
firewall_lan_udp_ports	[5353]	UDP ports allowed from the LAN (mDNS).
firewall_logging	low	ufw logging level.
firewall_docker_user_filter	true	Manage the DOCKER-USER block in /etc/ufw/after.rules that limits Docker-published ports to the LAN (ADR-0012).

Safety rails. Refuses first activation unless `magicbox_confirm_destructive=true`; always refuses when `preflight_controller_location` is `outside`.

Handlers. *Reload ufw* when the DOCKER-USER block changes.

Verification. `iptables -S DOCKER-USER` contains the LAN RETURN rule and the interface DROP rule. `sudo ufw status verbose` (manual); the controller play proves 9443 is reachable from the LAN.

Risks. A wrong `magicbox_lan_cidr` can lock out SSH from other subnets (console recovery: installation guide 8.3). Rules for ports no longer listed are not removed automatically; delete them with `sudo ufw delete <n>`. Ports published by containers bypass ufw's INPUT rules; the DOCKER-USER block restricts them to `magicbox_lan_cidr` instead. Setting `firewall_docker_user_filter: false` exposes them to any source that can route to MagicBox.

8. Role `docker`

Purpose. Install Docker Engine from Docker's apt repository with a verified signing key and a major-version pin, configure the daemon, and enable `containerd` and `docker` at boot (ADR-0002).

Variable	Default	Description
docker_major_version	"29"	apt preference pin 5:29.* for docker-ce, cli, rootless-extras.
docker_apt_key_url	https://download.docker.com/linux/<distro>/gpg	Signing key URL.
docker_apt_key_fingerprint	9DC858229FC7DD38854AE2D88D81803C0EBFCD88	Expected key fingerprint; mismatch aborts.
docker_apt_key_path	/etc/apt/keyrings/docker.asc	Keyring path referenced by the deb822 source.
docker_packages	docker-ce, docker-ce-cli, containerd.io, docker-buildx-plugin, docker-compose-plugin	Installed packages.
docker_conflicting_packages	docker.io, docker-doc, docker-compose, docker-compose-v2, podman-docker, containerd, runc	Removed only with confirmation.
docker_log_max_size	10m	json-file log rotation size.
docker_log_max_file	3	json-file rotated files kept.

`daemon.json` also sets the log option `tag` to the container name, which Alloy uses as the `container` label (ADR-0013).

Files. `/etc/apt/sources.list.d/docker.sources`, `/etc/apt/preferences.d/docker-ce.pref`, `/etc/docker/daemon.json` (validated with `dockerd --validate` before replacement; `live-restore` enabled).

Handlers. *Restart docker* (containers survive thanks to live-restore).

Verification. No `dockerd` TCP listener (`ss -Hltnp`).

Risks. Removing conflicting packages stops workloads that used them. A daemon restart briefly interrupts the Docker API. A major upgrade requires changing `docker_major_version` deliberately.

9. Role `portainer`

Purpose. Run Portainer CE with persistent data, HTTPS on 9443 using the MagicBox CA, plain HTTP disabled, and a pre-set admin password (ADR-0004, ADR-0005, ADR-0006). Container: `portainer`, image `portainer/portainer-ce:2.45.3`, restart policy `unless-stopped`, `network_mode: host`; mounts the Docker socket, the `portainer_data` volume, the certificate directory (read-only) and the password file (read-only).

Variable	Default	Description
portainer_image	portainer/portainer-ce	Image repository.
portainer_version	"2.45.3"	Image tag (LTS); also asserted against the API.
portainer_container_name	portainer	Container name.
portainer_https_port	9443	HTTPS port (keep in firewall_lan_tcp_ports).
portainer_data_volume	portainer_data	Persistent named volume mounted at /data.
portainer_base_dir	/opt/magicbox/portainer	Host base directory.
portainer_cert_dir	<base>/certs	TLS material (0700).
portainer_secret_dir	<base>/secrets	Admin password file (0700).
portainer_tls_mode	local_ca	local_ca or provided.
portainer_tls_dns_names	magicbox.local, magicbox	Certificate SANs and CA name constraints.
portainer_tls_ip_addresses	[]	Optional IP SANs/constraints. Keep empty: the IP is dynamic (DHCP).
portainer_tls_ca_common_name	MagicBox Local CA (magicbox)	CA subject CN.
portainer_tls_ca_valid_for	+3650d	CA lifetime.
portainer_tls_cert_valid_for	+397d	Server certificate lifetime.
portainer_tls_renew_before	+30d	Re-issue window before expiry.
portainer_tls_ca_fetch_dest	artifacts/magicbox-local-ca.crt	Controller copy of the CA for clients.
portainer_tls_provided_cert_src	empty	provided mode: server certificate path on the controller.
portainer_tls_provided_key_src	empty	provided mode: server key path (keep it vaulted).
portainer_tls_provided_ca_src	empty	provided mode: issuing CA path.

TLS tasks (`tls_local_ca.yml`). CA key (ECDSA P-384) and self-signed CA (`CA:TRUE, pathlen:0`, critical name constraints) created once; server key (ECDSA P-256), CSR and certificate (EKU `serverAuth`) signed by the CA and re-issued when within `portainer_tls_renew_before` of expiry or when the SANs change; CA fetched to the controller.

Handlers. *Restart portainer* when TLS material changes.

Verification. `GET https://magicbox.local:9443/api/system/status` from MagicBox with CA validation (retries 20 x 3 s) and an assertion that the reported version equals `portainer_version`. The controller play repeats the request over the LAN.

Risks. Portainer controls the Docker socket and is root-equivalent: every Portainer admin is root on MagicBox. Changing the vault password after first start does not change the admin password. Deleting `portainer_cert_dir` creates a new CA that clients must trust again. Deleting the `portainer_data` volume erases all Portainer settings; the playbook never does this.

10. Role `portainer_init`

Purpose. Initialise Portainer through its API: log in as admin with the vault password and ensure one local Docker environment named `magicbox_site_name` (`homelab`). It renames an existing local-socket environment, or creates one (ADR-0010). `tasks/api_login.yml` is reused by `stacks`.

Variable	Default	Description
portainer_init_api_user	**admin**	**Portainer user for API calls; password is portainer_admin_password.**

Prerequisites. The `portainer` role has run (Portainer up, CA present).

Verification. The environment exists and its URL is `unix:///var/run/docker.sock`.

Risks. Fails at login if the admin password was changed in the UI and not in the vault. Changing `magicbox_site_name` after stacks exist creates a new, empty environment; rename the old one in the UI first.

11. Role `stacks`

Purpose. Deploy the default services as Portainer standalone stacks in the site environment (ADR-0011). For each name in `stacks_enabled` it renders `roles/stacks/templates/<name>.yml.j2`, passes the secrets as stack environment variables, and creates the stack, or updates it when the compose file or environment differs or one of its containers is missing (e.g. an interrupted first deployment). Config files for `rabbitmq`, `ollama` (Open WebUI TLS), `observability`, `jenkins`, `kong`, `rustfs` and `registry` are written to `stacks_base_dir`; their checksum (`MAGICBOX_CONFIG_REVISION` label) forces a redeploy when they change.

Stack	Containers	Published ports	Volumes
rabbitmq	**rabbitmq**	**5672, 15672 (UI under /rabbitmq/)**	**rabbitmq_data**
postgresql	**postgresql**	**5432**	**postgresql_data**
ollama	**ollama, open-webui**	**11434 (API), 8444 (Open WebUI, HTTPS)**	**ollama_data, open_webui_data**
observability	**grafana, loki, alloy**	**3000 (Grafana HTTPS, under /grafana/)**	**grafana_data, loki_data, alloy_data**
rustfs	**rustfs**	**9100 (S3 API), 9101 (console under /rustfs/console/), both HTTPS**	**rustfs_data**
registry	**registry, registry-ui**	**5000 (registry API, HTTPS); UI on 127.0.0.1:5001, under /registry/**	**registry_data**
jenkins	**jenkins, jenkins-agent**	**8443 (Jenkins HTTPS, under /jenkins/)**	**jenkins_home, jenkins_agent_work**
dkron	**dkron**	**none: host networking, 127.0.0.1 8090 (UI/API, under /dkron/), 8946, 6868**	**dkron_data**
kong	**kong**	**80, 443 (host networking, ufw)**	**none (DB-less)**

Variable	Default	Description
stacks_enabled	**rabbitmq, postgresql, ollama, observability, rustfs, registry, dkron, jenkins, kong**	**Stacks to manage, in this order. Removing one does not delete it.**

Variable	Default	Description
stacks_base_dir	/opt/magicbox/stacks	Host directory for mounted config files.
stacks_rabbitmq_image	rabbitmq:4.3.7-management	RabbitMQ image.
stacks_postgresql_image	postgres:18.6	PostgreSQL image (newest major, supported to 2030).
stacks_ollama_image	ollama/ollama:0.40.12	Ollama image.
stacks_openwebui_image	ghcr.io/open-webui/open-webui:v0.12.5-slim	Open WebUI image, slim (no local RAG, ADR-0019).
stacks_grafana_image	grafana/grafana:13.2.5	Grafana image.
stacks_loki_image	grafana/loki:3.7.9	Loki image.
stacks_alloy_image	grafana/alloy:v1.21.2	Alloy image (Promtail replacement, ADR-0013).
stacks_rabbitmq_amqp_port	5672	Host port for AMQP.
stacks_rabbitmq_management_port	15672	Host port for the RabbitMQ UI/API (HTTP).
stacks_postgresql_port	5432	Host port for PostgreSQL.
stacks_ollama_port	11434	Host port for the Ollama API.
stacks_grafana_port	3000	Host port for Grafana (HTTPS).
stacks_openwebui_port	8444	Host port for Open WebUI (HTTPS, MagicBox certificate).
stacks_rabbitmq_user	magicbox	RabbitMQ default user.
stacks_rabbitmq_password	{{ vault_rabbitmq_password }}	RabbitMQ password (first start only).
stacks_postgresql_user	magicbox	PostgreSQL superuser.
stacks_postgresql_database	magicbox	Database created at first start.
stacks_postgresql_password	{{ vault_postgresql_password }}	PostgreSQL password (first start only).
stacks_grafana_admin_password	{{ vault_grafana_admin_password }}	Grafana admin password (first start only).
stacks_openwebui_admin_email	admin@magicbox.local	Open WebUI admin login name.
stacks_openwebui_admin_password	{{ vault_openwebui_admin_password }}	Open WebUI admin password (first start only; sign-up then disabled).
stacks_openwebui_secret_key	{{ vault_openwebui_secret_key }}	Signs Open WebUI sessions; changing it logs everyone out.
stacks_ollama_models	[]	Models to pull after deployment, e.g. ["llama3.2:3b"]; never deleted.
stacks_loki_retention	168h	Loki log retention.
stacks_jenkins_image	jenkins/jenkins:2.580.2-lts-jdk21	Base of the controller image built on MagicBox.
stacks_jenkins_agent_base_image	jenkins/ssh-agent:9.3.1-jdk21	Base of the agent image.
stacks_jenkins_docker_cli_image	docker:29.9.2-cli	Source of the Docker CLI and plugins copied into the agent.
stacks_jenkins_uv_image	ghcr.io/astral-sh/uv:0.13.6	Source of uv copied into the agent.
stacks_jenkins_plugins	11 plugins, pinned (incl. job-dsl for the app jobs)	Plugins baked into the controller image (name: version).
stacks_jenkins_port	8443	Host port for the Jenkins UI (HTTPS, MagicBox certificate).
stacks_jenkins_heap	1g	Controller JVM heap (-Xmx).

Variable	Default	Description
stacks_jenkins_memory_limit	2g	Controller container memory limit.
stacks_jenkins_agent_memory_limit	4g	Agent container memory limit (builds).
stacks_jenkins_agent_executors	2	Parallel builds on the agent.
stacks_jenkins_admin_password	{{ vault_jenkins_admin_password }}	Jenkins admin password (applied at every start).
stacks_jenkins_github_user	""	GitHub user name stored with the token (credential github).
stacks_jenkins_github_token	{{ vault_jenkins_github_token }}	Optional; becomes credential github.
stacks_jenkins_portainer_token	{{ vault_jenkins_portainer_token }}	Optional; becomes credential portainer-api-token.
stacks_jenkins_scan_interval	5m	Branch scan (GitHub polling) interval of the app jobs.
stacks_apps_dir	/opt/magicbox/apps	Host directory of apps.json (app name -> port), read by the app pipelines.
stacks_kong_image	kong:3.9.4	Kong OSS image (last open-source line, ADR-0015).
stacks_kong_http_port	80	Host port for HTTP; only redirects (301) to HTTPS. Keep in firewall_lan_tcp_ports.
stacks_kong_https_port	443	Host port for HTTPS (MagicBox certificate). Keep in firewall_lan_tcp_ports.
stacks_kong_upstream_timeout	600	Seconds Kong waits for an upstream (read/write).
stacks_kong_ollama_api_key	{{ vault_kong_ollama_api_key }}	API key required on /ollama/ (Kong key-auth, consumer ollama-client).
stacks_rustfs_image	rustfs/rustfs:1.0.4	RustFS image (ADR-0018).
stacks_rustfs_api_port	9100	Host port for the S3 API (HTTPS). Not 9000: it must stay closed.
stacks_rustfs_console_port	9101	Host port for the console (HTTPS, /rustfs/console/; also answers S3/admin calls).
stacks_rustfs_access_key	{{ vault_rustfs_access_key }}	RustFS root access key (every start).
stacks_rustfs_secret_key	{{ vault_rustfs_secret_key }}	RustFS root secret key (every start).
stacks_registry_image	registry:3.1.2	Docker registry image, CNCF Distribution (ADR-0020).
stacks_registry_ui_image	joxit/docker-registry-ui:2.6.0	Registry web UI (browse, delete tags), served at /registry/.
stacks_registry_htpasswd_image	httpd:2.4.69-alpine	Runs only htpasswd (bcrypt hash and check of the registry password).
stacks_registry_port	5000	Host port for the registry API (HTTPS, MagicBox certificate).
stacks_registry_ui_port	5001	Loopback HTTP port of the registry UI (reached through Kong).

Variable	Default	Description
stacks_registry_user	**magicbox**	**Registry login user (push and pull).**
stacks_dkron_image	**dkron/dkron:4.2.1**	**Dkron image (scheduler with UI, ADR-0022).**
stacks_dkron_port	**8090**	**Loopback HTTP port of the Dkron UI and API (reached through Kong at /dkron/).**
stacks_dkron_serf_port	**8946**	**Loopback Serf port (cluster membership).**
stacks_dkron_rpc_port	**6868**	**Loopback RPC/Raft port.**
stacks_dkron_user	**admin**	**Login user of /dkron/ (Kong basic-auth).**
stacks_dkron_password	**{{ vault_dkron_password }}**	**Login password of /dkron/; required by the Kong stack.**
stacks_registry_password	**{{ vault_registry_password }}**	**Registry password; the htpasswd file is rewritten only when it no longer matches.**

Internal data (`roles/stacks/vars/main.yml`, not meant to be overridden): `stacks_env` (environment per stack), `stacks_required_secrets`, `stacks_containers`, `stacks_images`, and for Jenkins the image tag (`stacks_jenkins_build_tag`, a hash of the build files), the image names, the Jenkins UID and the PKCS#12 keystore password, and the gateway routes (`stacks_kong_routes`, `stacks_kong_app_routes` built from `magicbox_apps`, `stacks_kong_all_routes`, `stacks_kong_port_redirects`, `stacks_reserved_ports`), and for the registry `stacks_registry_address` (`magicbox.local:5000`) and `stacks_registry_portainer_name` (`magicbox`).

Prerequisites. The host roles of the same run (Portainer, `homelab` environment, DOCKER-USER filter); the vault contains every stack password.

Verification. All stack containers running, and healthy where a healthcheck exists (up to 15 minutes, to cover first image pulls). PostgreSQL `pg_isready`. RabbitMQ `/api/overview` with the vault credentials. Ollama `/api/version`. Open WebUI over HTTPS with the MagicBox CA, sign-up disabled, admin login, and Ollama reached through it. Grafana `/api/health` over HTTPS with the MagicBox CA. Loki has a `container` label value `loki`, which proves Alloy -> Loki -> Grafana works.

Jenkins (ADR-0014). Ansible builds `magicbox/jenkins` and `magicbox/jenkins-agent` on MagicBox (tag = hash of the build files), writes `/opt/magicbox/stacks/jenkins/` (`casc/jenkins.yaml`, `secrets/` mode 0400 for UID 1000, `tls/jenkins.p12`), generates the controller -> agent SSH key and deploys the stack. The agent's sshd host key is also generated by Ansible and pinned in JCasC, so a recreated agent is still trusted. Jenkins is configured only by JCasC: user `admin`, no sign-up, no anonymous access, 0 executors on the controller, node `docker-agent` (label `docker`) over SSH. Verification: admin login over HTTPS with the MagicBox CA, anonymous access refused, agent online, `docker version` works on the agent as user `jenkins`. Credentials `github`, `portainer-api-token` and `registry` exist when their vault value is set. Settings managed by JCasC and changed in the UI are overwritten at the next start; jobs are created in the UI.

API gateway (ADR-0015). Kong OSS in DB-less mode, host networking, ports 80 (301 to HTTPS) and 443 with a copy of the MagicBox certificate. Ansible asserts that ufw allows both ports (firewall role of the same run), writes `/opt/magicbox/stacks/kong/declarative/kong.yaml` (root 0400, contains the API key) and `tls/`. Admin API (read-only) on `127.0.0.1:8001`, status on `127.0.0.1:8100`, Kong Manager off.

Route	Upstream (127.0.0.1)	Path to upstream	Auth
/, /portainer	**-**	**301 to /portainer/**	**-**
/rabbitmq	**-**	**301 to /rabbitmq/**	**-**
/portainer/	**Portainer 9443 (HTTPS)**	**stripped**	**Portainer login**
/jenkins	**Jenkins 8443 (HTTPS, --prefix=/jenkins)**	**kept**	**Jenkins login**
/grafana	**Grafana 3000 (HTTPS, serve_from_sub_path)**	**kept**	**Grafana login**

Route	Upstream (127.0.0.1)	Path to upstream	Auth
/rabbitmq/	RabbitMQ 15672 (management.path_prefix)	kept	RabbitMQ login
/ollama/	Ollama 11434 (API, no UI)	stripped	header/query apikey = vault_kong_ollama_api_key
/registry/	Registry UI 5001 (HTTP; proxies /v2 to the registry)	stripped	registry login (Basic, from the registry)
/dkron/	Dkron 8090 (UI /ui/, API /v1/)	stripped	basic-auth admin / vault_dkron_password (Kong)
/dkron, /dkron/ (exact)	-	301 to /dkron/ui/	-
/<name> (each app in magicbox_apps)	app <port> (HTTP)	stripped	app's own
/rustfs, /rustfs/	-	301 to https://magicbox.local:9101/rustfs/console/	-
/openwebui, /openwebui/	-	301 to https://magicbox.local:8444/ (ADR-0019)	-

Verification: every route whose container exists answers 200 through `https://magicbox.local/` with the MagicBox CA (Ollama with the key); `/ollama/` without the key gets 401; `/portainer`, `/rabbitmq` and `/registry` get a 301 adding the slash; `/rustfs` and `/openwebui` get a 301 to their own ports; HTTP gets a 301 to HTTPS. Apps registry assertions: unique names (lowercase, digits, hyphens, not a built-in route) and unique ports outside `stacks_reserved_ports`.

Apps (play 4, tag `apps`, ADR-0016, ADR-0017). In order: registry assertions and Kong routes (above); Jenkins must run with `CASC_JENKINS_CONFIG=/etc/jenkins/casc` (else "run `make deploy` first"); `/opt/magicbox/apps/apps.json`; `casc/apps.yaml` with one Job DSL `multibranchPipelineJob` per app (GitHub source, credential `github`, all branches, no pull requests, scan every `stacks_jenkins_scan_interval`); JCasC reload (crumb + admin login, no restart) when the file changed or a job is missing; every job exists; first scan of new jobs; report `deployed` / `registered, not deployed yet`. Each app's `Jenkinsfile` (template) takes the app name from the job name, reads its port from `apps.json`, and deploys the Portainer stack `<name>` (image `magicbox.local:5000/<name>:<commit>`, pushed to the registry with the Jenkins credential `registry` and pulled by Portainer, `127.0.0.1:<port>`). Removing an app from the registry removes its route and port, not its job or stack; job changes in the UI are overwritten.

Object storage (ADR-0018). RustFS, one data volume (single drive, no parity), root credentials from the vault. Ansible copies the MagicBox certificate to `/opt/magicbox/stacks/rustfs/tls/` (`rustfs_cert.pem`, `rustfs_key.pem`, UID 10001, 0400), so both ports serve HTTPS. The console uses the page's origin for its S3 and admin calls, so it is opened on its own port; the gateway path only redirects. S3 clients use `https://magicbox.local:9100`, path-style, region `us-east-1`. Verification: `/health` (9100) and `/rustfs/console/health` (9101) over HTTPS with the MagicBox CA; anonymous S3 request refused (403).

Docker registry (ADR-0020). CNCF Distribution, one data volume. Ansible copies the MagicBox certificate to `/opt/magicbox/stacks/registry/tls/` and keeps `auth/htpasswd` (bcrypt, root 0400) in sync with `vault_registry_password`: `htpasswd -vi` in `stacks_registry_htpasswd_image` checks it, `htpasswd -niB` rewrites it only on mismatch (password via stdin). The MagicBox engine trusts the CA through `/etc/docker/certs.d/magicbox.local:5000/ca.crt`, so the Jenkins agent pushes without insecure settings. Image names: `magicbox.local:5000/<name>:<tag>`. The registry is also added to Portainer (`tasks/portainer_registry.yml`, Registries > `magicbox`, custom, with login), so stack deployments pull its images; the password is sent on creation and again whenever the htpasswd file is rewritten (the API never returns it). Verification: `/v2/` over HTTPS with the MagicBox CA refuses anonymous access (401) and accepts the vault credentials, also through the UI's proxy (`127.0.0.1:5001/v2/_catalog`); the Docker engine logs in (throwaway client config, removed). Free disk space after deleting tags: `sudo docker exec registry registry garbage-collect /etc/distribution/config.yml`.

Scheduler (ADR-0022). Dkron, one server node, host networking with every listener on `127.0.0.1` (HTTP

`stacks_dkron_port`, Serf, RPC), data in `dkron_data`. Dkron OSS has no login: Kong's `basic-auth` (consumer `dkron-admin`) protects `/dkron/` and does not forward the credentials. HTTP jobs call apps on `127.0.0.1:<port>`. Verification: `/health` reports a leader; through the gateway `/dkron/health` answers with the login and `/dkron/v1/jobs` gets 401 without it.

Status. `make status` (`playbooks/status.yml`, `tasks/status.yml`) changes nothing: it lists each image of the enabled stacks as OK or MISSING and each container as OK, NOT OK (stopped, restarting, unhealthy) or MISSING, and fails when anything is not OK.

Risks.

- Passwords are applied only on first initialisation of each data volume. Changing a vault value later breaks login and verification until the service password is changed too.
- Stack environments (including passwords) are stored in Portainer's database.
- Edits made to these stacks in the Portainer UI are overwritten by the next run.
- RabbitMQ, PostgreSQL and Ollama use plain-text protocols on the LAN; Ollama has no authentication. Access is limited to the LAN by DOCKER-USER.
- Grafana, Open WebUI, Jenkins, Kong, RustFS and the registry use a copy of the Portainer certificate; `make deploy` renews it and refreshes the copies in the same run.
- First deployment downloads about 6 GB (Ollama about 3.4 GB, Open WebUI slim about 0.9 GB) and builds the Jenkins images.
- Open WebUI is reached on port 8444 (no sub-path support); `/openwebui` on the gateway only redirects. Slim image: document upload / RAG needs an embedding engine and a vector store, not configured.
- Jenkins agent: the mounted Docker socket is equivalent to root on MagicBox (ADR-0014). Build only private repositories; no pull requests from forks.
- Jenkins pipelines run against the host engine: `docker run -v $PWD:...` mounts host paths, not the agent workspace.
- Jenkins, Grafana and the RabbitMQ UI are served under `/jenkins/`, `/grafana/`, `/rabbitmq/` on their own ports too (old root URLs give 404).
- Kong: the direct ports stay open; Ollama on 11434 has no key. Kong's nginx master runs as root (binds 80/443). Upstream certificates are not verified (loopback only).
- RustFS: single drive, no redundancy: back up `rustfs_data`. Removing the stack keeps the volume; deleting `rustfs_data` deletes every bucket.
- Dkron: its login allows shell jobs as root in a host-network container; treat `vault_dkron_password` as an admin credential.
- Registry: anyone with `vault_registry_password` can push images that a pipeline may deploy. LAN clients must trust the MagicBox CA for `magicbox.local:5000`. Deleting `registry_data` deletes every image.

12. Role `os_update` (playbooks/update.yml)

Purpose. Keep the OS current: `apt update`, `apt upgrade` and `apt autoremove`. Docker CE updates stay within `docker_major_version` (apt pin); `live-restore` keeps containers running while dockerd restarts.

Variable	Default	Description
os_update_upgrade	safe	safe (apt upgrade, never removes packages) or full (apt full-upgrade).
os_update_autoremove	true	Remove packages that are no longer needed.
os_update_reboot	false	Reboot when /var/run/reboot-required exists.
os_update_reboot_timeout	900	Seconds to wait for MagicBox to return after the reboot.
os_update_services	avahi-daemon, docker	Must be running after the update; units not installed yet are skipped.

Verification. Every installed unit in `os_update_services` is running after the update (and after the reboot, if any). Before the first `make deploy`, Docker is not installed and is skipped.

Risks.

- A reboot stops Portainer and every stack for a few minutes; they restart automatically. Hence `os_update_reboot` is off by default and the play only reports a pending reboot.
- `full` may remove packages; review `apt list --upgradable` on MagicBox first.

13. Testing the recipes

Check	Command	Proves
Local checks	**make check**	**Playbooks parse with the example inventory; yamllint + ansible-lint production profile; no plaintext vault values or private keys tracked; manuals rebuilt; links, ADR registry, this reference covers all roles/variables/tags**
Container	**make test-container**	**Host and Portainer converge and changed=0 second run of site.yml in an Ubuntu 24.04 systemd container (arm64 or amd64 per workstation); tags stacks, apps skipped**
Container, everything	**make test-container ARGS="--stacks all"**	**The whole make deploy twice: every stack, app jobs, changed=0**
Container, some stacks	**make test-container ARGS="--stacks registry,jenkins,kong"**	**site.yml twice with stacks_enabled limited to the list**
Host	**make deploy (twice: changed=0), idempotency_check.py, --tags verify**	**Real behaviour on MagicBox**

The container test overrides `magicbox_expected_architecture`, `magicbox_lan_interface`, `magicbox_lan_cidr` and sets `preflight_manage_hostname: false`; it cannot prove LAN mDNS or reachability from other machines.

14. Changing the recipes

1. Edit the role; keep variables prefixed and documented in `defaults/main.yml`.
2. Update this reference (roles, variables, tags, risks) and, for significant decisions, add an ADR and list it in the registry and `CLAUDE.md`.
3. `make manuals check test-container` (with `ARGS="--stacks ..."` for stack changes).
4. Commit sources and rebuilt PDFs together.